

Certificación de Entrega

Entrega 0: Plan de proyecto

Introducción — Definición del proyecto

El objetivo de esta entrega es definir la idea general del proyecto considerando los módulos que luego deberán ser cumplidos y certificados.

Actualización: En caso de que se hagan modificaciones se deberán actualizar en todas las entregas.

Datos generales

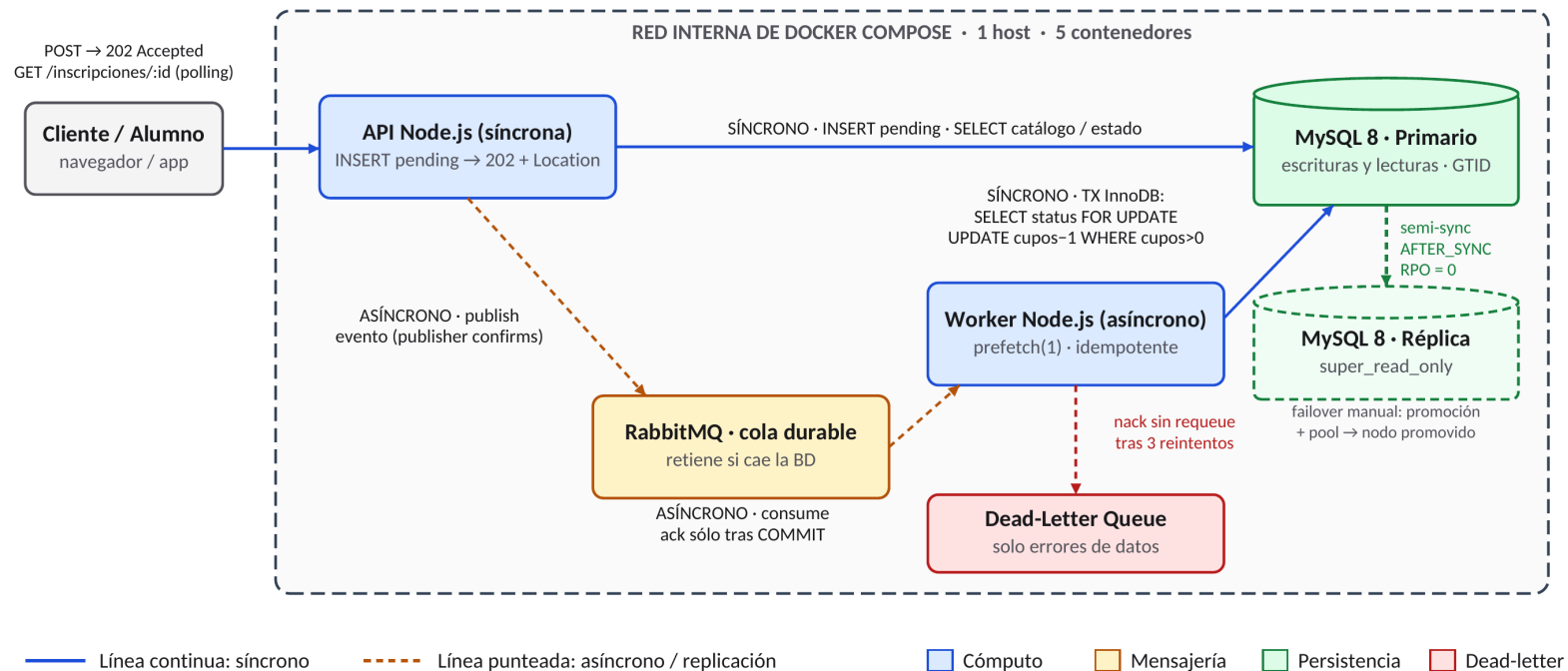
Grupo N°:	15
Tag / Release de esta entrega:	v0-plan-del-proyecto
Fecha de entrega:	06/09/2026

Proyecto

Área:	EdTech / Educación
Empresa o entidad asociada:	Universidad Blas Pascal — Proyecto de aplicación tecnológica de la cátedra Programación Distribuida y Componentes
Título:	Motor transaccional de inscripciones de alta concurrencia para una plataforma de e-learning
Breve descripción:	Sistema de inscripción a cursos con cupo limitado para una plataforma de e-learning, destinado a alumnos y administradores académicos. Está diseñado para absorber picos masivos de inscripciones simultáneas (el colapso típico de la apertura de un período, como el que sufren los sistemas de inscripción universitaria) sin saturar la base de datos, garantizando que nunca se asignen más cupos que los disponibles ni se inscriba dos veces al mismo alumno. Se implementa como una arquitectura distribuida de cinco contenedores orquestados con Docker Compose: una API Node.js síncrona que registra la intención y responde de inmediato, un broker RabbitMQ que amortigua el pico, un worker asíncrono que asigna los cupos de forma atómica e idempotente, y una capa de persistencia MySQL 8 en esquema primario-réplica con replicación semi-sincrónica, que elimina el punto único de falla del dato.
Dominio:	https://github.com/JoniMora/final-pdc

Certificación de Entrega

Infografía:



El problema

Al abrir el período de inscripción, miles de alumnos compiten por los mismos cupos en el mismo minuto. En una arquitectura síncrona, cada petición lee y escribe la misma fila de la base de datos: se agotan las conexiones, crece la latencia, el usuario reintenta y el sistema cae. Y entre leer y escribir se cuelan otras peticiones: más cupos asignados que los disponibles y dobles inscripciones.

La solución: Queue-Based Load Leveling

La API registra la intención (estado *pending*) y responde **202 Accepted** en milisegundos. RabbitMQ encola de forma durable. Un worker consume de a un mensaje y descuenta el cupo con un UPDATE condicional atómico dentro de una transacción InnoDB: la base de datos nunca ve el pico y no hay overbooking. El alumno consulta el resultado con GET /inscripciones/:id.

Garantías y criterio de aceptación

Idempotencia: verificación de estado bajo bloqueo de fila y UNIQUE(alumno, curso). **Tolerancia a fallos:** colas durables, publisher confirms, ack tras COMMIT, dead-letter queue; réplica MySQL semi-síncrona (RPO = 0 para lo confirmado) y failover manual con retención de mensajes en la cola. **Prueba de carga:** 500 POST concurrentes contra un curso de 10 cupos → exactamente 10 confirmados, 0 duplicados, 0 cupos negativos.

Certificación de Entrega

Descripción:

Qué hace el sistema y para quién. Es el motor de inscripciones de una plataforma de e-learning con cursos de cupo limitado. Los **alumnos** se inscriben y consultan el resultado; los **administradores académicos** definen cursos y cupos. No pretende tener más funcionalidades que un sistema de inscripción básico: pretende resolver bien un único problema difícil, que es sobrevivir al momento de apertura del período sin perder la consistencia de los cupos.

El problema. La apertura de inscripciones concentra en pocos minutos una demanda que el resto del cuatrimestre no existe. Es el mismo colapso que sufren los sistemas de inscripción universitaria (por ejemplo, SIU Guaraní) y las plataformas de venta de entradas. Una arquitectura síncrona clásica falla exactamente ahí: cada POST abre una conexión, lee el cupo, lo compara y escribe sobre la misma fila; las conexiones se agotan, la latencia crece hasta el timeout, el usuario reintenta y multiplica la carga. Peor aún, entre la lectura y la escritura se cuelan otras peticiones y el sistema asigna más cupos de los que existen o inscribe dos veces a la misma persona.

La solución. Se aplica el patrón *Queue-Based Load Leveling*, separando la parte rápida y paralela de la petición (registrar que un alumno *quiere* inscribirse) de la parte contenciosa y secuencial (decidir si *obtiene* el cupo). La API Node.js inserta la inscripción en estado *pending*, publica un evento en RabbitMQ y responde 202 Accepted con la URL de consulta. La cola, durable y ordenada, amortigua el pico. Un worker independiente consume los mensajes de a uno y, dentro de una transacción InnoDB, descuenta el cupo con `UPDATE cursos SET cupos = cupos - 1 WHERE id = ? AND cupos > 0`; si afecta una fila la inscripción pasa a *confirmed*, si afecta cero pasa a *rejected*. La operación es atómica y la garantía la da la base de datos, no la lógica de aplicación. El alumno consulta el resultado con GET /inscripciones/:id.

Garantías explícitas. *Idempotencia*: si el worker muere antes de confirmar el mensaje y RabbitMQ lo reentrega, el worker verifica primero, bajo bloqueo de fila, que la inscripción siga en *pending*; la restricción UNIQUE(alumno_id, curso_id) impide que un doble clic genere dos inscripciones. *Tolerancia a fallos en la mensajería*: cola y mensajes durables, publisher confirms, ack sólo después del COMMIT, reintentos acotados y dead-letter queue para mensajes que no pueden procesarse. *Tolerancia a la caída de la base de datos*: replicación semi-síncrona en modo AFTER_SYNC: el primario no confirma el COMMIT al cliente hasta que la réplica acusa haber recibido y persistido la transacción en su relay log, de modo que ninguna inscripción confirmada existe únicamente en el primario (RPO = 0 para el dato confirmado). Ante la caída del primario, la API responde 503 + Retry-After a las nuevas solicitudes, el worker suspende el consumo y RabbitMQ retiene los mensajes pendientes hasta que un administrador promueve la réplica y redirige el pool de conexiones; el procesamiento se reanuda sin pérdida ni duplicación, porque la verificación de estado bajo bloqueo de fila sigue vigente en el nodo promovido. *Límite declarado*: si la réplica no responde dentro del tiempo configurado (rpl_semi_sync_source_timeout), MySQL degrada a replicación asíncrona y el sistema queda temporalmente expuesto a la pérdida del último commit; ese evento se registra y se alerta.

Componentes y despliegue. Cinco contenedores en una red interna de Docker Compose sobre un único host: **api** (Node.js, único servicio expuesto al exterior), **rabbitmq** (cola principal y dead-letter queue), **worker** (Node.js), **mysql-primary** (MySQL 8 / InnoDB; recibe la totalidad de las escrituras y lecturas; binlog y GTID activados; volumen persistente propio) y **mysql-replica** (MySQL 8 en modo super_read_only; recibe los cambios del primario mediante replicación semi-síncrona y actúa como nodo de contingencia; volumen persistente propio). La API y el worker se comunican únicamente a través del broker y acceden al primario mediante un pool de conexiones cuyo destino se configura por variables de entorno. El diseño original (06/09/2026), con un único nodo MySQL, concentraba en ese contenedor un punto único de falla (SPOF): su caída detenía simultáneamente la API y el worker y exponía a la pérdida de las últimas transacciones confirmadas. La revisión del 17/09/2026, a pedido de la cátedra, incorpora la réplica y elimina el punto único de falla del dato; la conmutación (failover) es manual y forma parte del procedimiento operativo documentado en la Entrega 5. Todos los contenedores pueden escalarse horizontalmente, con excepción del primario, único por definición del esquema.

Fuera de alcance. Para mantener el proyecto realizable por dos personas en un cuatrimestre, quedan explícitamente fuera: telemetría de consumo de contenido, autenticación y autorización de usuarios, pasarela de pagos y un frontend elaborado. El cliente será mínimo: lo necesario para disparar inscripciones y consultar su estado, incluido el script de prueba de carga.

Anexo. El detalle de las cinco vistas del modelo 4+1 y el diagrama de despliegue se encuentran en la Entrega 1: <https://github.com/JoniMora/final-pdc/tree/main/docs/entrega-1> (informe.md, arquitectura.mmd, arquitectura.png, presentacion.pdf).

Certificación de Entrega

Planificación:

Gantt previsto de trabajo por módulo. Cada etapa cierra con una entrega certificable. Los períodos son aproximados y se ajustarán al calendario oficial de la cátedra.

Etapa / Entrega	Módulo	Período estimado	Entregable certificable
Entrega 0 · Plan de proyecto	Definición del proyecto	31/08 – 06/09	Plan, Gantt, épicas y reparto
Entrega 1 · Propuesta de arquitectura	Módulo 1 · Introducción a la programación distribuida	31/08 – 06/09	Vistas 4+1, diagrama de despliegue, informe
Entrega 2 · Infraestructura	Módulo 2 · Infraestructura	07/09 – 20/09	docker-compose con 5 contenedores, clúster MySQL primario-réplica semi-sincrónico, esquema, colas RabbitMQ (durable + DLQ)
Entrega 3 · Comunicación	Módulo 3 · Comunicación	21/09 – 04/10	API síncrona (catálogo, POST 202 + Location, GET estado) y publicación asíncrona con confirms
Entrega 4 · Middleware	Módulo 4 · Middleware	05/10 – 18/10	Worker: consumo prefetch(1), transacción InnoDB, UPDATE condicional, ack tras COMMIT, idempotencia
Entrega 5 · Seguridad y tolerancia a fallos	Módulo 5 · Seguridad y fallos	19/10 – 01/11	Caída simulada de broker, worker y primario MySQL; promoción de réplica; reentregas, reintentos, DLQ, durabilidad verificada
Entrega 6 · Escalabilidad	Módulo 6 · Escalabilidad	02/11 – 15/11	Prueba de carga (k6/autocannon): 500 POST vs 10 cupos → 10/0/0; escalado de workers y medición
Entrega 7 · Integración final	Cierre y defensa	16/11 – 29/11	Documentación consolidada, release final, presentación y defensa

Semana →	31/08	07/09	14/09	21/09	28/09	05/10	12/10	19/10	26/10	02/11	09/11	16/11	23/11
E0 · Plan de proyecto													
E1 · Arquitectura													
E2 · Infraestructura													
E3 · Comunicación													
E4 · Middleware													
E5 · Seguridad y fallos													
E6 · Escalabilidad													
E7 · Integración final													

Los nombres de los módulos 2 a 6 corresponden a la planificación del grupo y se alinearán con los nombres oficiales del programa de la cátedra en cuanto se confirmen; las fechas se corregirán con el calendario oficial de entregas.

Certificación de Entrega

Actividades:

Etapas / épicas que constituyen el proyecto y los sprints que las desarrollan. Cada sprint dura dos semanas y cierra con una entrega certificable, salvo el Sprint 0 (una semana).

Etapas / Épica	Sprint	Contenido	Entrega
Épica 1 • Base del proyecto	Sprint 0 • Plan y arquitectura	Plan de proyecto, vistas 4+1, diagrama de despliegue, repositorio y estructura de carpetas por entrega.	E0, E1
	Sprint 1 • Infraestructura	docker-compose.yml con api, rabbitmq, worker, mysql-primary y mysql-replica en red interna; esquema MySQL (cursos, alumnos, inscripciones con UNIQUE); cola durable y DLQ; healthchecks. Clúster primario-réplica: server_id distintos, binlog + GTID, usuario de replicación, réplica en super_read_only, plugins semi-sincrónicos en ambos nodos y verificación de Rpl_semi_sync_source_status = ON; script de inicialización (CHANGE REPLICATION SOURCE TO ... SOURCE_AUTO_POSITION = 1).	E2
Épica 2 • Servicio síncrono (API)	Sprint 2 • Contrato HTTP	GET /catalogo; POST /inscripciones con INSERT en estado pending, manejo de duplicado (1062) y respuesta 202 + Location + Retry-After; GET /inscripciones/:id.	E3
	Sprint 3 • Publicación asíncrona	Publicación del evento con publisher confirms; marcado failed + 503 si el broker no confirma; pruebas de integración API ↔ broker.	E3
Épica 3 • Motor asíncrono (Worker)	Sprint 4 • Consumo transaccional	Consumidor con prefetch(1) y ack manual; una conexión por transacción; UPDATE condicional del cupo; transición confirmed / rejected; ack sólo tras COMMIT.	E4
	Sprint 5 • Idempotencia y reintentos	SELECT ... FOR UPDATE del estado antes de tocar el cupo; manejo de 1213 / 1205 con reintentos acotados; nack sin requeue hacia DLQ.	E4
Épica 4 • Resiliencia	Sprint 6 • Fallos simulados	Caída y reinicio de rabbitmq y worker durante una ráfaga; verificación de que no se pierden mensajes ni se duplican cupos. Caída del nodo primario durante una ráfaga: la API responde 503 y el worker detiene el consumo sin derivar a la DLQ; promoción manual de la réplica (STOP REPLICATION; SET GLOBAL super_read_only = OFF); redirección del pool por variable de entorno y reinicio de api y worker; la cola se drena y el resultado cumple 10 / 0 / 0; medición de RPO (0) y RTO. Documentación de escenarios.	E5
Épica 5 • Validación y escalabilidad	Sprint 7 • Prueba de carga	Script k6/autocannon: 500 POST concurrentes contra un curso de 10 cupos; criterio 10 confirmados / 0 duplicados / 0 cupos negativos; escalado a N workers y medición de throughput y latencia.	E6
Épica 6 • Cierre	Sprint 8 • Integración y defensa	Consolidación de documentación de todas las entregas, release final, presentación y defensa oral.	E7

Certificación de Entrega

Participación:

Plan de trabajo de los integrantes. El reparto es por componente: cada integrante es responsable de un servicio de punta a punta (código, pruebas y documentación de su parte), con revisión cruzada de todo lo que se integra.

Integrante	Datos	Rol	Tareas asignadas
Jonathan Mora Colodrero	Legajo: 100462 Mail: jmoracolodrero@ubp.edu.ar	Infraestructura y motor asíncrono	Orquestación con Docker Compose (red interna, volúmenes, healthchecks). Configuración de RabbitMQ: cola durable, mensajes persistentes, dead-letter queue. Worker Node.js: consumo con prefetch(1) y ack manual, transacción InnoDB con UPDATE condicional del cupo, idempotencia mediante verificación de estado bajo bloqueo de fila, manejo de reintentos y errores de bloqueo, y distinción entre errores de datos (DLQ) y de infraestructura (requeue + pausa). Clúster MySQL primario-réplica semi-sincrónico, esquema, migraciones y procedimiento de failover. Sprints 1, 4, 5 y 6.
María Constanza Gigli	Legajo: 112075 Mail: margigli@ubp.edu.ar	Servicio síncrono y validación	API Node.js: catálogo, POST /inscripciones con registro del estado inicial y respuesta 202 + Location, GET /inscripciones/:id, manejo de duplicados y publicación del evento con publisher confirms. Cliente mínimo para disparar y consultar inscripciones. Diseño y ejecución de la prueba de carga (k6/autocannon), análisis de resultados y escalado de workers. Sprints 2, 3 y 7.
Ambos	—	Documentación y revisión	Redacción de cada entrega y de la presentación final, revisión cruzada de código (pull requests), escenarios de fallos simulados y defensa oral. Sprints 0 y 8.

Certificación de Entrega

Espacio para el docente

Nota: _____ / 10

Corregido por: *Completar*

Fecha de corrección: *dd/mm/aaaa*

Observaciones:

Espacio para comentarios del docente